

第八章

(第 3、4、5 题)

3. 根据下表给出的 4 个进程运行的起、止时刻, 指出每个进程对 P1—P2、P1—P3、P1—P4、P2—P3、P3—P4 中的两个进程是否并发运行?

进程	开始时刻	结束时刻
P1	1	7
P2	4	6
P3	3	8
P4	2	5

参考答案:

P1 和 P2 并发; P1 和 P3 并发; P1 和 P4 并发; P2 和 P3 并发; P2 和 P4 并发; P3 和 P4 并发。

4. 假设在 IA-32+Linux 系统中一个 main 函数的 C 语言源程序 P 如下:

```
1 unsigned short b[2500];
2 unsigned short k;
3 main( )
4 {
5     b[1000]=1023;
6     b[2500]=2049%k;
7     b[10000]=20000;
8 }
```

经编译、链接后, 第 5、6 和 7 行源代码对应的指令序列如下:

```
1 movw    $0x3ff, 0x80497d0    // b[1000]=1023
2 movw    0x804a388, %cx       // R[cx]=k
3 movw    $0x801, %ax          // R[ax]=2049
4 xorw    %dx, %dx            // R[dx]=0
5 div     %cx                  // R[dx]=2049%k
6 movw    %dx, 0x804a388       // b[2500]=2049%k
7 movw    $0x4e20, 0x804de20    // b[10000]=20000
```

假设系统采用分页虚拟存储管理方式, 页大小为 4KB, 第 1 行指令对应的虚拟地址为 0x80482c0, 在运行 P 对应的进程时, 系统中没有其他进程在运行, 回答下列问题。

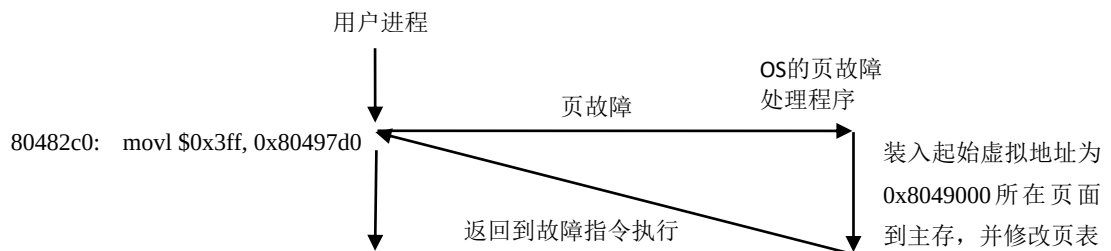
- (1) 对于上述 7 条指令的执行, 是否可能在取指令时发生缺页异常?
- (2) 执行第 1、2、6 和 7 行指令时, 在访问存储器操作数的过程中是否会发生页故障或其他什么问题? 哪些指令中的问题是可恢复的? 哪些指令中的问题是不可恢复的? 分别画出第 1 行和第 7 行指令所发生故障的处理过程示意图。
- (3) 执行第 5 条指令时会发生什么故障? 该故障能否恢复?

参考答案:

因为 Linux 在初始化时, 通常将所有段的段基址都设为 0, 所以, 虚拟(逻辑)地址就是线性地址。

(1) 第 1 行指令的虚拟地址为 0x80482c0, 其线性地址为 0x80482c0, 而页大小为 4KB, 因此, 第 1 行指令不是一个页面的起始地址, 所以, 在执行第 1 行指令前面的指令时, 上述 7 条指令被同时装入主存, 因而, 上述 7 条指令的执行过程中都不会在取指令时发生缺页。

(2) 执行第 1 行指令过程中，数据访问时会发生缺页，但是是可恢复的故障。因为，对于地址为 0x80497d0 的 b[1000] 的访问，是对所在页面（起始地址为 0x8049000，是一个 4KB 页面的起始位置）的第一次访问，因而对应页面不在主存。当 CPU 执行到该指令时，检测到缺页异常，即发生了页故障，此时，CPU 暂停 P 对应的用户进程的执行，将控制转移到操作系统内核，调出内核中的“页故障处理程序”执行。在页故障处理程序中，检查是否地址越界或访问越权，显然这里没有发生越界和越权情况，故将地址 0x80497d0 所在页面从磁盘调入内存，处理结束后，再回到这条 movl 指令重新执行，此时，再访问数据就没有问题了。处理过程如下图所示。



对于第2行指令的执行，数据访问时会发生缺页，因为在地址0x804a388位于起始地址为0x804a000的页面中，对地址0x804a388的访问是该页的第一次访问。与第1行指令的数据访问一样，是可恢复的故障。

对于第6行指令的执行，数据访问时不会发生缺页，因为地址0x804a388所在的页面（其起始地址为0x804a000的页面）已经在主存中。但是，该指令的执行会破坏原来存放在地址0x804a388中的变量k的值。

对于第7行指令的执行，数据访问时很可能会发生页故障，而且是不可恢复的故障。显然，a[10000]并不存在，不过，C编译器生成了对应的指令“movl \$0x4e20,0x804de20”，其中的地址0x804de20偏离数组首地址0x8049000已达 $2 \times 10000 + 2 = 20002$ 个单元，即偏离了4、5个页面，很可能超出了可读写数据区范围，因而当CPU执行该指令时，很可能发生地址越界或访问越权。若是这样的话，CPU就通过异常响应机制转到操作系统内核，即调出内核中的页故障异常处理程序执行。在页故障处理程序中，检测到发生了地址越界或访问越权，因而页故障处理程序发送一个“段错误”信号（SIGSEGV）给用户进程，用户进程接受到该信号后就调出一个信号处理程序执行，该信号处理程序根据信号类型，在屏幕上显示“段故障（segmentation fault）”信息，并终止用户进程。处理过程如下图所示。



(3) 因为这里忘记给 k 赋值，且 k 是未初始化的变量，所以 k 在.bss 节中。通常.bss 节中的变量初值都自动设为 0，因而，执行第 5 条指令时，会发生“整除 0”故障，该故障是不能恢复的。

5. 若用户程序希望将字符串“hello, world!\n”中的14个字符显示在标准输出设备文件stdout上，则可以使用系统调用write对应的封装函数write(1, “hello, world!\n”,14)，在IA-32+Linux系统中，可以用以下机器级代码（用汇编指令表示）实现。

```
1  movl  $4, %eax      //调用号为4，送EAX
2  movl  $1, %ebx      //标准输出设备stdout的文件描述符为1，送EBX
```

```

3  movl $string, %ecx      //字符串“hello, world!\n”的首地址等于string的值，送ECX
4  movl $14, %edx          //字符串的长度为14，送EDX
5  int   $0x80             //系统调用

```

针对上述机器级代码，回答下列问题或完成下列任务。

- (1) 执行该段代码时，系统处于用户态还是内核态？为什么？执行完第 5 行指令后的下一个时钟周期，系统处于用户态还是内核态？
- (2) 第 5 行指令是否属于陷阱指令？执行该指令时，通过 5 种类型（中断门、系统门、系统中断门、陷阱门和任务门）门描述符中的哪种类型门描述符来激活异常处理程序？对应的中断类型号是多少？对应门描述符中的字段 P、DPL、TYPE 的内容分别是什么？根据对应门描述符中的段选择符取出的 GDT 中的段描述符中的基地址、限界、字段 G、S、TYPE（包含 A）、DPL、D 和 P 分别是什么？
- (3) 详细描述第 5 行指令的执行过程。

参考答案：

- (1) 因为该段代码执行的是用户程序的功能，因此，在执行第 5 行软中断指令（int \$0x80）进行系统调用之前，系统处于用户态，执行完第 5 行指令后的下一个时钟周期，系统从用户态陷入到内核态执行。
- (2) 第 5 行指令属于陷阱指令。执行该指令时，通过系统门（system gate）描述符来激活异常处理程序，对应的中断类型号是 128。对应门描述符中的 P=1（Linux 总是把 P 设成 1），DPL=3，TYPE=1111B。根据对应门描述符中的段选择符取出的 GDT 中的段描述符应该是内核代码段对应的段描述符（Linux 下设定为表 7.2 中所示的内核代码段，因为执行系统调用后应调出内核代码执行），所以，其基地址为 0，限界为 FFFFFFFH，G=1，S=1，TYPE=1010，DPL=0，D=1，P=1。（参看 7.5.1 节表 7.2）有关 TYPE 的含义参见下图。

type<8时

十进制值	TYPE				说明
	E	W	A		数据段
0	0	0	0	0	只读
1	0	0	1	0	只读、已访问
2	0	1	0	0	读写
3	0	1	1	0	读写、已访问
4	1	0	0	0	只读、向下扩展
5	1	0	1	0	只读、向下扩展、已访问
6	1	1	0	0	读写、向下扩展
7	1	1	1	0	读写、向下扩展、已访问

type>=8时

十进制值	TYPE				说明
	C	R	A		代码段
8	1	0	0	0	只执行
9	1	0	0	1	只执行、已访问
10	1	0	1	0	执行、可读
11	1	0	1	1	执行、可读、已访问
12	1	1	0	0	只执行、一致
13	1	1	0	1	只执行、一致、已访问
14	1	1	1	0	执行、可读、一致
15	1	1	1	1	执行、可读、一致、已访问

- (3) 第 5 行指令的执行过程如下：

① 确定中断类型号为 128（0x80），从 IDTR 指向的 IDT（参考 8.3.2 节中关于 IDT 初始化的描述）中取出第 128 个表项，这个表项实际上就是 Linux 初始化时在 IDT 的第 128 项中设定的系统门描述符，其中 P=1，DPL=3，TYPE=1111B，段选择符为 0x60，指向 GDT 中的内核代码段描述符（kernel_code）。Linux 全局描述符表（GDT）如下。

Linux 全局描述符表	段选择符	Linux 全局描述符表	段选择符
null	0x0	TSS	0x80
reserved		LDT	0x88
reserved		PNPBIOS 32-bit code	0x90
reserved		PNPBIOS 16-bit code	0x98
not used		PNPBIOS 16-bit data	0xa0
not used		PNPBIOS 16-bit data	0xa8
TLS #1	0x33	PNPBIOS 16-bit data	0xb0
TLS #2	0x3b	APMBIOS 32-bit code	0xb8
TLS #3	0x43	APMBIOS 16-bit code	0xc0
reserved		APMBIOS data	0xc8
reserved		not used	
reserved		not used	
kernel code	0x60 (__KERNEL_CS)	not used	
kernel data	0x68 (__KERNEL_DS)	not used	
user code	0x73 (__USER_CS)	not used	
user data	0x7b (__USER_DS)	double fault TSS	0xf8

② 根据 IDT 中的段选择符，从 GDTR 指向的 GDT 中取出相应的段描述符，得到对应异常处理程序或中断服务程序所在段的 DPL、基地址等信息。Linux 下即为内核代码段，所以 DPL 为 0，基地址为 0。

③ 将当前特权级 CPL（CS 寄存器最低两位，00 为内核特权级，11 为用户特权级）与段描述符中的 DPL 比较。若 CPL 小于 DPL，则产生 13 号异常（#GP）。因为，Linux 内核代码段的 DPL 总是 0，因此不管怎样都不会发生 CPL 小于 DPL 的情况。

④ 检查是否发生了特权级变化，即判断 CPL 是否与相应段描述符中的 DPL 不同。因为执行第 5 行指令时处于用户态，因此 CPL=3，而 DPL=0，故需要从用户态切换至内核态，以使用内核对应的栈。通过以下步骤完成栈的切换：

- a) 读 TR 寄存器，以访问正在运行进程的 TSS 段；
- b) 将 TSS 段中保存的内核栈的段选择符和栈指针分别装入寄存器 SS 和 ESP，然后在内核栈中保存原来的用户栈的 SS 和 ESP。

⑤ 将第 5 行后面一条指令的逻辑地址写入 CS 和 EIP，以保证内核程序处理后回到下条指令执行。

⑥ 在当前栈中保存 EFLAGS、CS 和 EIP 寄存器的内容。

⑦ 将 IDT 中的段选择符（0x60）装入 CS，将 IDT 中的偏移地址装入 EIP，它指向内核代码段中的系统调用处理程序 system_call 的第一条指令。

这样，从下一个时钟开始，就执行系统调用处理程序 system_call 的第一条指令。在内核完成系统调用服务后，通过执行最后一条指令 IRET，以回到第 5 行指令的下一条指令继续执行。